A Type-Confined Termination Ordering for Gödel's T
Exact comparison, a ranked path-order restriction, and Bachmann–Howard height

28 October 2026
Working manuscript

Abstract

We construct an effective, context- and substitution-stable termination ordering for the literal lambda formulation of Gödel's T. It is a restriction of recursive StarHorpo by inclusion of the finite types occurring in a term. The restriction is essential for the ordinal analysis. A finite target-observation interpretation decides the recursive comparison despite the unbounded extra-argument lists generated by its Select clause. Every original beta, case and recursion rule decreases strictly. On a disjoint finite ranked ground signature the restriction is exactly multiset recursive path ordering. Every individual descending rank is below the Bachmann–Howard ordinal and their supremum is that ordinal. The lower argument builds literal native-recursion realizers for finite-type ordinal iteration; the upper argument uses a finite-type arithmetic accessibility proof and the published inductive-norm bound for ID_1. The ranked first-order restriction is distinguished from uncurrying the native case functional, and descending height is distinguished from a linear order type. The construction does not decide comparison of the numerical Wilken–Weiermann canonical assignments.

Contents

1 A syntactic ordering question **2**

2 The recursive comparison **2**

3 An exact finite comparison algorithm **4**

4 Confinement, stability and exact ranked RPO **7**

5 The original reductions decrease **8**

6 An individual Bachmann–Howard upper bound **9**

7 Native finite-type rank iteration **11**

8 Interpretation and limitations **13**

1 A syntactic ordering question

The normalization of Gödel's T does not by itself give a convenient syntactic ordering of its terms. Wilken and Weiermann ask for extensions of recursive path ordering adapted to the finite-type calculus and anticipate Bachmann–Howard strength [1, Introduction]. A related question asks whether comparisons between their particular ordinal-vector assignments can be decided. Those are different specifications. The present paper concerns the syntactic ordering specification.

There are three requirements whose proofs should not be conflated. A recursive description of a relation must actually give a terminating comparison algorithm. A well-founded relation must orient the original typed rewrite rules and survive insertion in contexts and substitution. Finally an ordinal-height theorem must concern that same relation, rather than a better-behaved auxiliary ordering. We establish these requirements separately and then combine them.

1.1 Terms and the first-order embedding

Simple types are generated by a base type o and arrows. The type level is $\mathrm{lv}(o) = 0$ and $\mathrm{lv}(\sigma \to \tau) = \max(\mathrm{lv}(\sigma) + 1, \mathrm{lv}(\tau))$. The native constants are $0 : o$, $S : o \to o$, $D^\tau : o \to \tau \to \tau \to \tau$, and $R^\tau : o \to (o \to \tau \to \tau) \to \tau \to \tau$. Application is left associative. The non-contextual rules are

$$(\lambda x.A)B \to A[x := B], \quad D^\tau 0AB \to A, \quad D^\tau (Sn)AB \to B,$$

$$R^\tau 0AB \to B, \qquad R^\tau (Sn)AB \to A\, n\, (R^\tau nAB).$$

Substitution is capture avoiding. The reduction relation includes their instances in every well-typed term context.

All these constants are ranked nodes of arity zero, even when their simple type is functional. Encode an application UV literally as $@_{\sigma,\tau}(U, V)$, with declaration $[\sigma \to \tau, \sigma] \to \tau$. Denote this encoding by μ. It commutes with substitution by structural induction. No normalization, eta expansion or uncurrying is part of μ.

Independently adjoin a finite ranked signature $\mathcal{F}$ with declarations $[o, \ldots, o] \to o$. These are actual ranked nodes; a ternary $d \in \mathcal{F}$ is not the curried native constant D^o. Give all heads multiset status. The typed application heads form one precedence-equivalence class, all native constants form one strictly higher class, and the finitely many heads of $\mathcal{F}$ have a fixed strict total precedence above those classes.

Definition 1.1 (Type support)**.** $\mathrm{Ty}(s)$ is the set of literal subterm types and binder types of s, closed under taking type subexpressions. It is a finite subtype-closed set.

The declaration of a ranked ground node contributes only its argument and result types. Its absent curried counterpart is not a subterm type.

2 The recursive comparison

We specialize the ordinary, bottom-free, no-meta-variable fragment of Kop's recursive StarHorpo [2, Definition 5.19]. The details below fix the fragment, so later comparisons do not hide an iterative reachability oracle.

An ordinary ranked head $f(\vec{a})$ has its fixed base arguments. A marked head $f^\star(\vec{a}; \vec{e}) : \tau$ retains them, may have a finite list of additional arguments, and may be retyped to an

output type τ. Retyping never changes an argument's type. Marking an ordinary term passes through its initial lambda spine and marks its first functional head. If there is no such head it is undefined.

Definition 2.1 (Simultaneous special application)**.** For $q : \rho_1 \to \cdots \to \rho_n \to \tau$, define $q\langle a_1, \ldots, a_n\rangle$ as follows. With no arguments the result is q. If at least n initial lambdas are present, consume exactly n and substitute their arguments simultaneously. Otherwise consume the $k < n$ initial lambdas. If the remaining *original* head is functional, substitute in its existing arguments, mark it if needed, and append all remaining $n - k$ arguments at once. If that head is a variable, the operation is undefined.

For example, with $T = o \to o$, $h : T$ a native head and $y : o$,

$$(\lambda x : T.x)\langle h, y\rangle \text{ is undefined}, \qquad (\lambda x : T.x)\langle h\rangle\langle y\rangle = h^\star(; y).$$

Thus full special application is not obtained by iterating unary tables.

Write $\succeq_*$ for weak comparison of equal-typed terms and $s \succ_* t$ for $s^\star \succeq_* t$. The all-Mul ordinary clauses are:

1. $x \succeq_* x$.
2. $\lambda x.s \succeq_* \lambda x.t$ if $s \succeq_* t$.
3. $f(\vec{s}) \succeq_* g(\vec{t})$ if $f \approx g$ and their equally long argument multisets admit a bijection of weak comparisons.
4. $f(\vec{s}) \succeq_* t$ if $f^\star(\vec{s}) \succeq_* t$.
5. A marked source $P = f^\star(\vec{s}; \vec{e}) : \tau$ weakly dominates t if an argument q from either list has a suffix type $\rho_1 \to \cdots \to \rho_n \to \tau$ and $q\langle P_{\rho_1}, \ldots, P_{\rho_n}\rangle \succeq_* t$. The case $n = 0$ is included.
6. $P \succeq_* \lambda x.t$ if, with fresh x, the marked source with x appended to its extras weakly dominates t.
7. $P \succeq_* g(\vec{t})$ if f is strictly above g and the source retyped to each target-argument type weakly dominates that argument.
8. The same conclusion holds when $f \approx g$, the *base* argument multiset of P strictly exceeds the target multiset under the generalized weak/strict multiset test, and those whole-source premises also hold.

These are respectively Var, Abs, Fun, Put, Select, F-Abs, Copy and Stat. All derivations are finite. Strict multiset comparison retains weakly matched occurrences and replaces or deletes a nonempty source part, with every new occurrence strictly below an occurrence of that part. Extras are selectable but do not enter the Stat base multiset.

The published strong-pair theorem for this recursive relation supplies transitivity, compatibility, well-foundedness and substitution stability under a well-founded precedence and compatible statuses [2, Theorems 5.32, 5.37 and 5.39]. We use these published properties; they do not supply the new finite comparator or the calibration proved here.

3 An exact finite comparison algorithm

Fix ordinary inputs (s, t). Let K be their subtype closure, V their free names, and B the maximum nesting depth of a lambda in t. Recursive target premises are opened subterms of t. A target lambda is opened at a fresh typed port. Use B port slots; at context Γ retain only opened observations with remaining lambda depth at most $B - |\Gamma|$. Write O^{Γ}_{σ} for this finite set of observations of type σ. Contexts extend by adding previously unused slots, keeping their types.

Lemma 3.1 (Finite closure)**.** *Every type requested by a recursive comparison belongs to K, every source head is a head of the original source, and every target observation and opening context belongs to the finite family just described.*

Proof. Select uses a suffix of the type of an existing argument; all its argument and result types are subtypes of that type. The retyped copies introduce only these types. Copy and Stat request target-argument types. F-Abs uses the domain and codomain of the observed target lambda. Substitution does not change a body-node type. These exhaust the ways to create a type. Marking and Select preserve the source head; Copy creates no new recursive source with its target head. The only fresh port is introduced while descending a target lambda, so used ports plus remaining target depth never exceeds B. The free names of generated sources lie in $V \cup \Gamma$. □

We now give finite domains on which special application and source comparison can be computed without bounding the size of generated sources.

Definition 3.2 (Natural finite domains)**.** At base type, a value at Γ consists, at each extension Δ, of an availability bit and weak/strict subsets of O^{Δ}_{o}. At arrow type σ, add for each nonempty suffix decomposition $\sigma = \rho_1 \to \cdots \to \rho_n \to \tau$ a monotone full tuple table

$$D^{\Delta}_{\rho_1} \times \cdots \times D^{\Delta}_{\rho_n} \longrightarrow D^{\Delta}_{\tau}.$$

All argument and result types are proper subtypes, so this is a recursion on types. Values are ordered pointwise. Weakening $a \mapsto a|_{\Theta}$ retains the components at extensions of Θ. Restrict the raw finite products to values satisfying every equation

$$\mathrm{App}_{\Delta}(a, \vec{v})|_{\Theta} = \mathrm{App}_{\Theta}(a|_{\Theta}, \vec{v}|_{\Theta}) \qquad (\Delta \subseteq \Theta).$$

Only these hereditary natural values form D^{Γ}_{σ}.

Lemma 3.3 (Effective complete lattices)**.** *The domains in 3.2 are finite complete lattices, effectively enumerable from the inputs. Weakening preserves their joins and meets.*

Proof. The claim holds for finite products of bits and powersets. At an arrow, the lower-type domains are already constructed. Enumerate all tables and retain the monotone ones satisfying the finitely many naturality equations. Pointwise joins and meets of a family of such tables remain monotone and natural, because weakening is a coordinate projection and preserves lower-type joins and meets. The bottom and top families satisfy the equations as well. This proves the inductive claim and gives a finite construction. □

For every possible source head f, output type $\tau \in K$, and context, introduce a monotone constructor table $M^{\Gamma}_{f,\tau}(\vec{b}, E)$. The tuple $\vec{b}$ has the fixed base types of f; E is a down-set in the disjoint union of the finitely many typed domains at Γ. Require constructor coherence

$$M^{\Gamma}_{f,\tau}(\vec{b}, E)|_{\Delta} = M^{\Delta}_{f,\tau}(\vec{b}|_{\Delta}, {\downarrow}\{a|_{\Delta} : a \in E\}). \tag{3.1}$$

The coherent monotone families form a finite complete lattice, by the same pointwise argument. The down-set represents the values of *all* extra arguments, without preserving their irrelevant order or multiplicity.

At the syntactic level, irrelevance of extra order and repetition can be checked directly. Let $\equiv_E$ be congruence generated by permuting or repeating marked extras, keeping every distinct extra and every base argument. It preserves types, free names, substitution, retagging and the original-spine operation. Induction on a finite comparison derivation transports it across $\equiv_E$: Select still finds its corresponding argument, F-Abs appends the same fresh variable, and Copy and Stat inspect only the unchanged head and base list. Var, Abs, Fun and Put use their induction premises. The reverse implication follows by symmetry. This explains the set convention but does not prove semantic compression by itself; adequacy below proves that the finite down-set abstraction also loses no comparison.

3.1 The fixed-point equations

For a marked state $(f, \vec{b}, E)$ use the following equations for its weak observation bits. All data are first weakened to the observation context.

1. Select chooses a value a from $\vec{b}$ or E and a compatible suffix decomposition. Apply its one full tuple table to the corresponding retyped M values of the whole state; use the resulting weak bit.

2. Copy tests strict head precedence and the weak bits for every target argument of the correspondingly retyped whole state.

3. Stat tests equivalent heads, the finite generalized strict multiset test on $\vec{b}$, and the same whole-source bits.

4. F-Abs opens the target at a fresh port, weakens existing inputs, and appends the port value to the extra down-set.

The availability bit is true. A marked state's strict bits are empty. Its full tuple table appends the entire supplied tuple to E and consults the result-type constructor. An unmarked $f(\vec{b})$ has weak bits from Fun or its marked state, strict bits from that marked state's weak bits, and tuple tables that mark the head and append all supplied arguments. Variables have their literal weak bits, no strict bits, and undefined nonempty tuple tables. Undefined means the whole bottom value, including availability, not just an empty weak observation set.

A lambda has matching bits against lambdas obtained from its opened body. For its tuple table, scan the *original* lambda spine. Each consumed binder is recorded as protected. With arguments left at a protected-variable head return bottom; at a functional head append all remaining arguments simultaneously; at an unprotected free name consult its full table. With no arguments left interpret the residual term in the updated environment. This is a structural helper, not extensional beta normalization.

These equations define an operator F on the finite constructor lattice.

Lemma 3.4 (Structural interpretation)**.** *At a natural coherent constructor family, interpretation and the original-spine helper are monotone, commute with weakening and capture-avoiding substitution, and interpret simultaneous special application exactly.*

Proof. Induct jointly on the finite source and the original-spine helper. At a variable use its assigned natural value; the protected variable case is constantly bottom. At a functional head use monotonicity and (3.1). At a lambda consume a binder with a fresh name outside V,

the port supply and substitution ranges, and apply the induction hypothesis to the original body. Its literal lambda matching bits are retained independently of its tuple behavior. A free substituted variable uses its full tuple table, which has already recorded its own original spine. If the supplied tuple ends within a lambda spine the two constructions are ordinary simultaneous substitution; otherwise the original residual head determines either the same marked head with the same extras or the same undefined case on both sides. Naturality at the variable case is precisely the tuple equation in 3.2; at a functional head it is (3.1). No unary-currying identity is used. □

Lemma 3.5 (A finite monotone operator)**.** *F preserves natural coherent families and is monotone. Iteration from bottom reaches its least fixed point in finitely many steps.*

Proof. Every rule test is positive in weak and strict bits. Application uses monotone tables, weakening is a projection, and taking a down-set is monotone. The structural interpretation is monotone by 3.4. At extension Δ every observation equation consults only weakened inputs and M^{Δ}. For tuple components use

$$\downarrow(E \cup \{v_i\})|_{\Delta} = \downarrow(E|_{\Delta} \cup \{v_i|_{\Delta}\}),$$

which follows from monotonicity of weakening and inclusion of the original image on each side. This proves coherence and tuple naturality of $F(M)$. The ascending bottom iteration stabilizes because the lattice is finite. □

Lemma 3.6 (Adequacy)**.** *At the fixed point, a weak or strict observation bit is true exactly when the corresponding finite recursive comparison is derivable.*

Proof. For soundness relate a value a at Γ to a concrete term q with $\mathrm{FV}(q) \subseteq V \cup \Gamma$. Require that all its bits at all extensions are derivable comparisons from q, and that each full tuple table preserves the relation on related, defined, concrete argument tuples. Only bottom relates to an undefined result. This relation is downward closed, by induction on types. Bottom relates to every permitted defined term.

Prove by stage induction that each constructor approximation preserves this relation. Select picks a concrete base or extra argument related to the selected value; a value below an actual extra's value remains related to that same extra. The tuple property and 3.4 give its literal premise, so the Select rule gives the conclusion. Copy and Stat use their displayed premises. F-Abs uses a fresh port, fresh for the source by the support condition; its enlarged context supplies precisely the opened premise. Tuple outputs are the corresponding concrete marked head with all supplied arguments appended. The structural fundamental lemma is proved by the same source/helper induction as 3.4. Thus the limiting interpretation is sound.

For completeness induct on a finite comparison derivation. Var, Abs and Fun are their structural equations. Put inserts a marked weak bit into an unmarked weak set, and strictness inserts that bit into its strict set. Select uses the exact full-tuple operation; every actual extra is in its downward closure, so it is not lost. Copy, Stat and F-Abs are their equations. The context/type invariants in 3.1 retain every target premise. These exhaust the bottom-free, no-meta-variable rules. □

Theorem 3.7 (Effective comparison)**.** *Both recursive comparisons on ordinary bottom-free inputs are decidable.*

Proof. Construct the finite domains and constructor lattice, compute its fixed point, interpret s at the empty context, and test the root observation t. Adequacy proves both answers correct. No generated-source size bound or numerical Select fuel is part of the algorithm. □

4 Confinement, stability and exact ranked RPO

Definition 4.1. For equal outer types put

$$s \succeq_c t \iff \mu(s) \succeq_* \mu(t) \ \wedge \ \mathrm{Ty}(t) \subseteq \mathrm{Ty}(s), \qquad s \succ_c t \iff \mu(s) \succ_* \mu(t) \ \wedge \ \mathrm{Ty}(t) \subseteq \mathrm{Ty}(s).$$

Lemma 4.2 (Free-variable containment)**.** *Every weak comparison $s \succeq_* t$ satisfies* $\mathrm{FV}(t) \subseteq \mathrm{FV}(s)$.

Proof. Use simultaneous induction for weak comparison and strictness through marking. Var is immediate; Abs removes the common opening variable. Fun distributes target arguments to source arguments. Put preserves free variables. Copy and Stat have a whole-source premise for each target argument. In Select the special application has free variables only from its selected argument and the supplied whole-source copies, all already in the source. F-Abs adds a fresh opening variable to the extras, and binding it in the target removes exactly that possible addition. No remaining rule applies. □

Lemma 4.3 (Exact support substitution)**.** *For a finite type-preserving substitution γ,*

$$\mathrm{Ty}(s\gamma) = \mathrm{Ty}(s) \cup \bigcup_{x \in \mathrm{FV}(s)} \mathrm{Ty}(\gamma(x)),$$

where absent entries mean the variable itself.

Proof. Every old node retains its result or binder type, and every new node is in an inserted image. Conversely every type in an inserted image occurs at an actual free occurrence. Structural induction gives both inclusions. □

Theorem 4.4 (Stable effective pair)**.** *The confined pair is decidable, compatible and transitive; its weak part is reflexive and its strict part is well-founded. Both are stable under well-typed substitution and every literal context.*

Proof. Decidability follows from 3.7 and a finite support test. The published StarHorpo strong-pair theorem gives the underlying relational properties. Support inclusions compose, so restriction preserves transitivity and both mixed compatibilities. Restriction preserves well-foundedness. A context adds the same set of types to each side. For substitution use 4.2 and 4.3: every image type added on the target side is also added on the source side. Combine this with the published substitution property and $\mu(s\gamma) = \mu(s)\gamma_\mu$. □

Theorem 4.5 (Exact ranked restriction)**.** *On the first-order $\mathcal{F}$-algebra, $\succ_c$ is precisely multiset RPO for the selected precedence. Its weak part is RPO together with recursive multiset equality.*

Proof. Every type support is $\{o\}$. Between first-order endpoints, an unmarked source uses only Var, Fun or Put; an initially marked source uses only Select, Copy or Stat. Every argument is ground, so Select has zero supplied arguments. No extras or lambdas are generated. The recursive comparisons therefore stay within this first-order fragment.

Induct on a weak derivation, interpreting an unmarked comparison as weak RPO and an initially marked comparison as strict RPO. Var is equality. Fun is a permutation of weak argument comparisons: either all are recursively equal, or the multiset strictly decreases. Put includes strictness into weakness. Select is the subterm clause, Copy is the precedence clause and Stat is the strict multiset clause. Conversely induct on an RPO/equality derivation. Use Var/Fun for recursive multiset equality, zero-argument Select for the subterm clause, Copy

for precedence, and Stat for multiset decrease. Stat's side comparisons with target arguments follow by Select from the matched source arguments, using Put to include any strict match weakly. There is no derivation path through a hidden higher-order intermediate. □

Example 4.6 (A ranked comparison)**.** Take a ranked constant $a : o$, heads $f : [o, o] \to o$ and $g : [o] \to o$, and precedence $f > g > a$. Then $f(a, g(a)) \succ_c g(g(a))$. The marked f source selects its second argument with zero supplied arguments, giving the strict subterm comparison with $g(a)$. Copy at $f > g$ uses that whole-source premise to construct the target $g(g(a))$. All type supports are $\{o\}$, so confinement imposes no additional test. The theorem identifies precisely the same derivation with ordinary multiset RPO; no curried application node is involved.

5 The original reductions decrease

Lemma 5.1 (Template grafting)**.** *For an ordinary template A and $p \succeq_* q$ of the same type, $A[x := p] \succeq_* A[x := q]$, even if p contains marked subterms.*

Proof. Induct on the ordinary template. Its x leaf is the hypothesis; other variables are Var. Abs uses fresh bound names, and Fun uses the induction comparisons with identity argument matching. This is not a claim about arbitrary marked contexts. □

Lemma 5.2 (Beta and cases)**.** *Every original beta and case step decreases under $\succ_c$.*

Proof. For a beta redex put $P_\rho = @^\star(\lambda x.A, B) : \rho$. Select on its second base argument gives $P_\sigma \succeq_* B$. Template grafting gives $A[x := P_\sigma] \succeq_* A[x := B]$. Select on the first argument with its one self-copy now gives $P_\tau \succeq_* A[x := B]$, hence strict beta orientation.

For $D^\tau uAB$, select the outer second argument to obtain B. For A, select the outer first argument with one self-copy; the resulting marked inner application still has A as its second base argument, selected with zero arguments. These comparisons hold for every test u, and in particular orient the two original rules. The right-hand-side type support is included in that of the original redex in each case. □

Lemma 5.3 (Explicit application contexts)**.** *If ordinary $a \succ_* b$, replacing a by b in any ordinary context gives another strict comparison.*

Proof. Strict comparison implies weak comparison: peel common initial lambdas and apply Put at the marked head. At a ranked context head, Stat matches all unchanged arguments reflexively and the changed argument strictly. Each whole-source side premise follows by zero-argument Select of the corresponding source argument and its reflexive or weak comparison. Abs handles a lambda context. Induct on context depth. The support condition is preserved by union, so the result also holds for $\succ_c$. □

For the successor recursor write $H = o \to \tau \to \tau$, $U_1 = R^\tau(Sn)$, $U_2 = U_1 A$, $L = U_2 B$, and $L' = R^\tau nAB$. Use the following marked families, with their original heads and base lists:

$$P_\rho = @^\star(U_2, B) : \rho, \qquad C_{2,\rho} = @^\star(U_1, A; P_\tau) : \rho,$$

$$C_{1,\rho} = @^\star(R^\tau, Sn; C_{2,H}, C_{2,\tau}) : \rho, \quad C_{0,\rho} = (R^\tau)^\star(; C_{1,o}, C_{1,H}, C_{1,\tau}) : \rho.$$

Lemma 5.4 (Recursor orientation)**.** *Both original recursor rules decrease strictly under $\succ_c$.*

Proof. The zero rule selects the outer second argument B. For the successor rule, three Select prefixes carry a weak comparison from $C_{0,\tau}$ back through $C_{1,\tau}, C_{2,\tau}$ to P_τ. They select U_2, then U_1, then R^τ, with all self-arguments supplied simultaneously at the displayed types.

From $C_{0,H}$ select successively $C_{1,H}$, $C_{2,H}$ and A to obtain $C_{0,H} \succeq_* A$. From $C_{0,o}$ select $C_{1,o}$ and Sn; since $Sn \succ_* n$ by outer Select, this yields $C_{0,o} \succeq_* n$. At type τ, the extra chain reaches P_τ. Context closure of $Sn \succ_* n$ gives $L \succ_* L'$, or $P_\tau \succeq_* L'$; prefixing the extra chain gives $C_{0,\tau} \succeq_* L'$. Two Copy clauses at $R^\tau > @$ first give $C_{0,\tau\to\tau} \succeq_* An$, then $C_{0,\tau} \succeq_* AnL'$. Prefix the three outer Selects. All types in the contractum already occur in the redex; this proves confinement. The reasoning is uniform in arrow or base τ. □

Corollary 5.5. *Every original T reduction, in every well-typed context and substitution instance, strictly decreases under $\succ_c$.*

6 An individual Bachmann–Howard upper bound

For a well-founded relation write $\rho_R(x) = \sup_{xRy}(\rho_R(y) + 1)$. The ordinal-analysis input is the following published theorem: for a positive arithmetic inductive operator, every individual membership atom provable in ID_1 has inductive norm below BH; for the accessibility operator this norm equals the descending rank [5, Lemma 6.5.2, Definition 9.1.2 and Theorems 9.5.5–9.5.6]. We give the finite-type accessibility and edge interpretation needed to use it.

Fix a finite subtype-closed set S. Number the marked heads by $k \in \mathbb{N} \cup \{\omega\}$, where unmarked heads have label ω. Retain finite base and extra lists and output annotations in S, and add $\bot_\sigma$ for each $\sigma \in S$ and ordinary auxiliary application. Use the all-Mul numbered reduction of [2, Section 5.3], restricted to S: marking ω to any finite label; Select at $k+1$ applying a chosen argument to retyped self-copies at k; Copy to a lower-precedence unmarked head with self-arguments at k; Mul replacement of a nonempty part of the base list by marked arguments; creation of an abstraction with one extra and label decreased by one; reductions in ranked arguments and lambda bodies; function-side reduction in auxiliary application; beta; and reduction to bottom. Every instance is well typed in S. There is no argument-side monotonicity for auxiliary application.

The primary equivalence permutes equivalent heads and their appropriate base arguments, preserving labels, types and extra-list order. With de Bruijn binders its classes are finite and computable: the shape is unchanged and only finitely many S-heads and permutations occur. Work on the least-code representatives. Let H be the induced one-step relation and $R = H^+$. Both are arithmetic; equivalence-only motion has been quotiented out.

Lemma 6.1 (Arithmetic accessibility proof)**.** *For this fixed S, ID_1 proves that every well-typed auxiliary term is accessible under R.*

Proof. Use two positive arithmetic operators: $I(x)$ says every R successor of x is in I, and $J(M)$ says every direct multiset replacement successor of the finite multiset M is in J. They are separate unnested inductive definitions, with well-typedness included in both defining formulas. They refer to the arithmetic relations R and $\mathrm{DM}(R)$, not to one another's inductive predicates. Disjoint numerical tags combine them into one positive arithmetic definition, as permitted in ID_1. For the finitely many types in S, define arithmetic computability predicates by $\mathsf{C}_o(t) = I(t)$ and

$$\mathsf{C}_{\sigma\to\tau}(t) \iff \forall u(\mathsf{C}_\sigma(u) \to \mathsf{C}_\tau(tu)).$$

The type recursion is finite; no new inductive definition at an arrow is used. Typedness is conjoined in each clause. Quantification over arbitrary computable arguments in the following

induction invariants abbreviates the finite disjunction of these typed formulas over S; it is an actual first-order formula, rather than a satisfaction predicate for all simple types.

First prove by type induction that a non-lambda is computable when all its direct reducts are; variables and bottoms are computable; computability implies I and is closed under H and R. At base type use closure and transitivity of R. At an arrow, a non-lambda application tu reduces only in its function or to bottom, so the lower-codomain induction applies. To infer $I(t)$ from $I(tx)$ for a fresh variable, use I-induction: every descendant t' of t has $t'x$ as a descendant of tx. This proves inverse application.

For a lambda whose computable instances $v[x := u]$ are all computable, choosing $u = x$ gives $I(v)$. Use I-induction on v, retaining the universal instance premise. The reducts of $(\lambda x.v)u$ are its beta contractum, function-side body reducts and bottom; all are computable by the premise, the induction hypothesis and the bottom case. Hence the application and the lambda are computable.

Next, accessibility of the members implies $J(M)$. To prove closure under multiset union, use outer J-induction on one side and inner J-induction on the other. Label removed occurrences by their origin and assign each replacement to a removed dominator; each component is unchanged or decreases, and one decreases. Then I-induction on a gives $J(\{a\})$, since every replacement is a finite collection of descendants of a. Finite unions give the general claim.

Prove that a numbered constructor with computable arguments is computable by the lexicographic induction

$$(\text{head class}, \text{base multiset}, k, \text{extra multiset}).$$

This means finite external induction on head classes, J-induction on the base multiset with all later parameters universally quantified, induction on finite k followed by the ω case, and J-induction on extras. Marking decreases k. Select applies a computable selected argument to the lower-label computable self-copies. Copy uses lower k then a lower head class. Mul decreases the base multiset because an initially marked argument is an R descendant of its unmarked argument; its new head may reset to ω because the base parameter is earlier. Abstraction creation decreases k before adding any extra, and uses the lambda fact just proved. Argument reductions decrease the appropriate multiset. Bottom is computable. These are all auxiliary rule cases. Finally structural induction under a computable substitution makes every S-term computable, hence in I. All induction invariants are first-order formulas of ID_1. □

Lemma 6.2 (Arithmetic simulation)**.** *There is a computable translation* tr *such that* $s \succ_c t$ *implies* $\mathrm{tr}(s) R \,\mathrm{tr}(t)$ *in the fixed universe* $S = \mathrm{Ty}(s)$.

Proof. Relate a recursive source q to a numerical source Q at budget n when unmarked heads correspond to label ω and every marked head corresponds to a finite label at least n, with identical type, base and extra structure. At quotient classes require a witnessing raw representative. The relation is arithmetic, decreases with n, and commutes with substitution.

If q and a_i correspond at budget $n+l$, where l arguments are supplied, the auxiliary application $QA_1 \cdots A_l$ reduces to a source corresponding at budget n to the defined $q\langle a_1, \ldots, a_l\rangle$. Consume the same original lambda spine by beta; at its functional residual head mark ω if needed, then create an abstraction and beta-contract once for each remaining argument. Each such operation appends that argument and spends one label. Definedness was tested on the original residual head, before substituted binders are inspected. All types are domain or suffix types of q.

Induct on a finite recursive derivation. Var needs budget zero; Abs and Fun use the maximum child budget and context or quotient permutation. Put marks ω at a sufficiently

large finite label. Select with l arguments uses child budget plus $l+1$: its one Select step makes ordinary applications to unchanged lower-label self-copies, and the preceding angle simulation supplies the exact premise. F-Abs uses child budget plus one. Copy uses one plus the largest child budget. Stat chooses its finite retained/changed occurrence assignment; changed source arguments are initially marked at the preceding label and retained ones are unchanged, then child certificates finish every target position. Its budget is one plus their maximum. All choices and paths are finite arithmetic certificates. An original strict comparison starts by a nonempty marking step, so its complete simulated path is positive. Every requested type is in $\mathrm{Ty}(s)$ by confinement and 3.1. □

Theorem 6.3 (Individual upper bound)**.** *Every descending $\succ_c$ rank is strictly below* BH*.*

Proof. For a fixed source use $S = \mathrm{Ty}(s)$. Every confined descendant stays in S. The two preceding lemmas give an ID_1 proof of accessibility of $\mathrm{tr}(s)$ and a rank-monotone edge simulation. The published inductive-norm extraction gives $\rho_{\succ_c}(s) \leq \rho_R(\mathrm{tr}(s)) < \mathsf{BH}$. Only an individual membership atom is needed; no uniform notation for all ranks or numerical bound on comparison time is inferred. □

7 Native finite-type rank iteration

Restrict $\succ_c$ to closed terms of each type and write ρ for its rank. Restriction cannot increase a rank. Its type-stable closed contexts retain the properties already proved. Put $\sigma = o \to o$. All these ranks are countable because the syntax is countable and the relation is well founded; no upper calibration is needed for that fact.

Lemma 7.1 (A native carrier)**.** *Let $P : B \to B$, $U : B$, and $K \succeq_c P$ be closed. With $W = R^B 0(\lambda n : o.K)U$, one has $W \succ_c U$, and*

$$W \succ_c T \quad \Longrightarrow \quad W \succ_c PT.$$

Proof. Use the four marked families from the recursor proof with test 0, step $\lambda n.K$, and base U. Their three Select prefixes lift weak comparisons from the marked native R^B to the marked outer W. The extra chain reaches U. At output $B \to B$, select the step-function extra, supplying a ground self-copy; follow its original marked application extras to $\lambda n.K$ and consume its unused ground binder. The result is $K \succeq_c P$. At output B, the extra chain reaches $W^\star$, to which the assumed $W^\star \succeq_* T$ is appended. Copy at $R^B > @$ now constructs PT, and prefixing the three Select steps gives the assertion. Every target type already occurs in P, U, K, W; the assumed confinement is preserved. □

Translate semantic ordinal types by $|\iota| = \sigma$ and arrows homomorphically. At the semantic base use the countable ordinals, with all set functions at arrows. Say $t \Vdash_\iota \alpha$ if $\rho(t) \geq \alpha$, and $t \Vdash_{A \to D} f$ if every closed realizer u of x satisfies $tu \Vdash_D f(x)$. Countable suprema at arrows are pointwise.

Lemma 7.2 (Realization closure)**.** *Realization is backward closed under $\succ_c$, downward closed in the semantic pointwise order, and closed under countable suprema realized by one fixed term. The typed lambda fundamental lemma holds.*

Proof. Induct on semantic type. At base use the rank inequality and closure of ordinal upper bounds under countable suprema. At arrows apply the induction claim to every realizing argument and use context closure. For the lambda case substitute the given realizers and use the oriented beta step. Variables and application are exactly the environment and arrow definitions. □

Let $E_\iota(i) = i$ and $E_{A\to D}(i) = \lambda x : |A|.E_D(i)$. For $B = |A|$, $\eta = B \to B$, and $Q_\eta = \lambda a : \eta.\lambda b : \eta.a$, put

$$K_i = Q_\eta P E_{A\to A}(i), \quad W_i = R^B 0(\lambda n : o.K_i)U.$$

Lemma 7.3 (Iteration inequalities)**.** *$W_i \succ_c U$. If $i \succ_c j$ then $W_i \succ_c W_j$ and $W_i \succ_c PW_j$.*

Proof. The nonempty beta path gives $K_i \succeq_c P$. The carrier lemma yields $W_i \succ_c U$. Literal context closure gives $W_i \succ_c W_j$; apply the carrier lemma to obtain the last inequality. The fixed template adds the same types on both sides, and PW_j adds only the already present $B \to B$ and B. □

Theorem 7.4 (Full-prefix ordinal iteration)**.** *There is an effective closed native term I_A realizing*

$$(\alpha, f, x) \longmapsto f^\alpha(x), \qquad f^0(x) = x, \quad f^{\beta+1}(x) = f(f^\beta(x)), \quad f^\lambda(x) = \sup_{\xi<\lambda} f^\xi(x).$$

No monotonicity of f is required.

Proof. Take $\mathsf{I}_A = \lambda i : \sigma.\lambda p : \eta.\lambda u : B.R^B 0(\lambda n : o.Q_\eta p E_{A\to A}(i))u$. For closed realizers P, U, prove that the single W_i realizes every orbit value x_β for $\beta \le \rho(i)$. Induct externally on $\rho(i)$ and internally on β. Zero follows from $W_i \succ_c U$. For $\gamma + 1 \le \rho(i)$, the rank recursion supplies $i \succ_c j$ with $\rho(j) \ge \gamma$. The outer induction gives $W_j \Vdash x_\gamma$; then $PW_j \Vdash x_{\gamma+1}$ and 7.3 transports it backwards. At a nonzero limit the same W_i realizes every earlier stage; countable-supremum closure gives that full-prefix supremum. This uses all earlier stages, not just a cofinal subsequence of a nonmonotone orbit. Finally beta-orient the displayed lambda term and apply 7.2. □

Lemma 7.5 (Zero, successor and a finite cap)**.** *Closed native terms realize 0, right successor and ω. For every $P : B \to B, U : B$, a closed term lies strictly above every $P^n U$.*

Proof. $Z = \lambda z : o.z$ realizes zero. The selector $C_B(s,t) = (\lambda a : B.\lambda b : B.a)st$ lies strictly above t by outer Select; hence $\mathbf{s} = \lambda a : \sigma.C_\sigma(Z, a)$ realizes right successor. Put $\mathrm{Cap}(P,U) = R^B 0(\lambda n : o.P)U$. The carrier lemma gives the base $\mathrm{Cap} \succ_c U$ and iterates the implication to $P^n U$. Thus $\mathrm{Cap}(\mathbf{s}, Z)$ has rank at least ω and realizes it. □

7.1 Matching the published calibration

The preceding primitives realize ordinal addition by iterating right successor, multiplication by iterating $x \mapsto x + \alpha$ from zero, and $\alpha \mapsto \omega^\alpha$ by iterating $x \mapsto x\omega$ from one. For clarity this is an effective literal compilation, not an existence claim about arbitrary semantic constants. With $\mathbf{w} = \mathrm{Cap}(\mathbf{s}, Z)$ and $\mathbf{1} = \mathbf{s}Z$, the terms are

$$\mathbf{add} = \lambda a\, b.\mathsf{I}_\iota b\, \mathbf{s}\, a, \qquad \mathbf{mul} = \lambda a\, b.\mathsf{I}_\iota b\, (\lambda x.\mathbf{add}\, x\, a)\, Z,$$

$$\mathbf{pow} = \lambda a.\mathsf{I}_\iota a\, (\lambda x.\mathbf{mul}\, x\, \mathbf{w})\, \mathbf{1}.$$

Their omitted binder types are all σ. The full-prefix iteration theorem proves their stated ordinal meanings, including at limits. They therefore realize

$$\mathrm{Fix}(f)(\zeta) = f^\omega(\zeta + 1), \quad \mathrm{Next} = \mathrm{Fix}(\alpha \mapsto \omega^\alpha), \quad \epsilon = \mathrm{Next}(0).$$

The literal realizer of Fix is $\lambda f\, z.\mathsf{I}_\iota \mathbf{w}\, f\, (\mathbf{s}z)$; apply it to **pow** for Next and then to Z for ϵ. Let $A_0 = \iota$, $A_{n+1} = A_n \to A_n$. For $h : A_{l+1}$ and $h_j : A_j$, define $B_l : A_{l+2}$ by

$$B_l h\, h_l \cdots h_1 = \mathrm{Fix}(\lambda \alpha^\iota.h^\alpha h_l \cdots h_1\, 0).$$

For $l = 0$ use $h^{\alpha}0$; for $l > 0$ its first iteration is the instance at A_l. Its realizer replaces $h^{\alpha}h_l$ by $\mathsf{I}_{A_l}\alpha hhh_l$ and applies the remaining arguments and Z; at $l = 0$ use $\mathsf{I}_{\iota}\alpha hZ$. Thus every B_l has a finite native term obtained by explicit lambda abstraction and application. Define

$$d_0 = \epsilon, \qquad d_1 = \mathrm{Next}(\epsilon), \qquad d_{l+2} = B_l B_{l-1} \cdots B_0 \,\mathrm{Next}\, \epsilon.$$

These are the precise finite-type extractors of [3, Definitions 2.4–2.5, 2.10, 2.12, 2.14; Theorem 6.1 and Corollary 6.2]. Their helpfulness hypotheses are supplied by [4, Definition 5.1 and Theorem 5.10]. More precisely the semantic types are the full finite endotype hierarchy, iteration takes the pointwise supremum of the *entire* earlier prefix, the fixed-point functional starts at $\zeta + 1$, and the base pair is $(\mathrm{Next}, \varepsilon_0)$. These are exactly the hypotheses of that calibration, including the terminal zero argument in B_l; a base pair starting at ω is not substituted for it.

Let $a_0 = 0$, $a_{n+1} = \Omega^{a_n}$, with $\Omega = \omega_1$. The published comparison identifies d_n with $\psi_A(a_n)$ and $\sup_n a_n = \varepsilon_{\Omega+1}$. Here ψ_A is the hull collapse generated from the displayed base pair using addition, ω-exponentiation and available smaller collapses. The upper bound uses Pohlers's Veblen-closed convention ψ_B. Simmons's comparison is cofinal agreement of those conventions, not pointwise equality. Their monotonicity and the cofinal tower sequence give equal final suprema. At the limiting argument, each finite hull derivation uses finitely many collapse parameters, so is already present below one earlier bound. Hence

$$\sup_n d_n = \sup_n \psi_A(a_n) = \psi_B(\varepsilon_{\Omega+1}) = \mathsf{BH}.$$

The cited calibration and helpfulness theorems supply this ordinal-analysis step; the native carrier and iterator proofs above supply its new term interpretation for the particular confined relation.

Theorem 7.6 (Exact descending height)**.** *Every individual $\succ_c$ rank is below* BH*, and*

$$\sup_s(\rho_{\succ_c}(s) + 1) = \mathsf{BH}.$$

An effective family of closed native terms of outer type $o \to o$ already has cofinal ranks.

Proof. Replace every primitive in the finite expression d_n by its native realizer and apply the typed fundamental lemma. This gives an effective finite term $t_n : \sigma$ with $\rho(t_n) \geq d_n$. There are only finitely many type instances in each expression, and confinement has been checked for every carrier, context and beta step. Consequently $\mathsf{BH} = \sup_n d_n \leq \sup_n \rho(t_n)$, bounded above by the global height. The individual upper theorem gives the reverse inequality. □

8 Interpretation and limitations

The construction has an exact ranked first-order RPO restriction and a literal original-T subsystem. It does not identify $d(n, a, b)$ with the native curried $D^o nab$. The latter encoding has additional application-prefix structure, and exact RPO agreement under it is a stronger question. Nor does the height theorem assert a total ordering or the order type of any linear extension.

The comparison algorithm is finite but can be extremely large: the domains contain full monotone tuple tables. No practical complexity bound, executable implementation, or optimal rank notation is asserted. Finally, this ordering is not defined by comparing the numerical Wilken–Weiermann canonical vectors. Its decidability leaves the residual arithmetic problem of [6, 7] separate.

References

[1] Gunnar Wilken and Andreas Weiermann. Derivation lengths classification of Gödel's T extending Howard's assignment. *Logical Methods in Computer Science* **8**(1:19) (2012), 1–44. doi:10.2168/LMCS-8(1:19)2012. Official article and full text. The 2012 revised article is the version used throughout.

[2] Cynthia Kop. *Higher Order Termination: Automatable Techniques for Proving Termination of Higher-Order Term Rewriting Systems.* PhD thesis, Vrije Universiteit Amsterdam, 2012. IPA Dissertation Series 2012-14; ISBN 978-94-6203-164-7. Author-hosted copy with corrections. Definition 5.19, printed pp. 117–118, is the recursive StarHORPO definition used here.

[3] Harold Simmons. A comparison of two systems of ordinal notations. *Archive for Mathematical Logic* **43** (2004), 65–83. doi:10.1007/s00153-003-0177-z. Definitions 2.4–2.5, 2.10, 2.12 and 2.14 specify the extractor interface; Theorem 6.1 and Corollary 6.2 provide its calibration.

[4] Harold Simmons. Fruitful and helpful ordinal functions. *Archive for Mathematical Logic* **47** (2008), 677–709. doi:10.1007/s00153-008-0102-6. Definition 5.1 and Theorem 5.10 supply the extractor and helpfulness results used by the comparison calibration.

[5] Wolfram Pohlers. *Proof Theory: The First Step into Impredicativity.* Universitext. Springer, Berlin and Heidelberg, 2009. doi:10.1007/978-3-540-69319-2. The cited upper analysis uses Definition 9.1.2 and Theorems 9.5.5–9.5.6; collapse conventions must be matched through the cited comparison theorems.

[6] *The Wilken–Weiermann Comparison Problem: Foundations and a Guide to the Results.* Companion working manuscript, 2026. Included manuscript.

[7] *Actual-Source Arithmetic and Undecidability Boundaries for Canonical Comparison.* Companion working manuscript, 2026. Included manuscript.